

لماذا يُنصح بتعيين قيم صريحة لعناصر الـ Enum في بعض الحالات؟

لضمان التوافق مع قواعد البيانات (Database mapping) أو واجهات برمجة التطبيقات (APIs)، وتجنب تغيير القيم الافتراضية إذا تم إضافة أو حذف عناصر جديدة في المنتصف مستقبلاً. ماذا يحدث إذا قمت بتعيين قيمة لعنصر Enum تتجاوز نطاق النوع الأساسي (Underlying type)؟

سيحدث خطأ في وقت الترجمة (Compile-time error) لأن القيمة لا تتناسب مع المساحة التخزينية المحجوزة للنوع. ما هو الغرض من كلمة virtual عند استخدامها مع الخصائص (Properties)؟

تسمح للـ Classes الوارثة (Derived classes) بعمل override لتعديل أو تغيير سلوك هذه الخاصية. لماذا لا يمكنك عمل override لخاصية أو دالة تحمل الكلمة sealed؟

لأن sealed تنهي سلسلة التوريث لهذا العنصر، وتُستخدم لحماية المنطق البرمجي (Business logic) من التعديل غير المتوقع في الطبقات الأدنى. ما هو الفرق الرئيسي بين الأعضاء الـ static والـ object members؟

الـ static ينتمي للـ Class نفسه ويتم مشاركته على مستوى الذاكرة مرة واحدة ولا يتطلب أخذ نسخة (Instance)، بينما الـ object member ينتمي للنسخة (Instance) المنشأة، وكل Object يمتلك نسخه الخاصة من البيانات. هل يمكنك عمل Overload لجميع الـ Operators في C#؟ ولماذا؟

لا. بعض المعاملات مثل &&, ||, =, الـ dot . لا يمكن عمل Overload لها في C# لتجنب كسر القواعد الأساسية للغة وطريقة تقييم الشروط (Short-circuiting). متى يجب أن تفكر في تغيير النوع الأساسي (Underlying type) للـ Enum؟

عند الحاجة لتوفير الذاكرة (Memory Optimization) إذا كان هناك ملايين السجلات (استخدام byte بدلاً من int)، أو عند التعامل مع أنظمة خارجية تتطلب أنواع بيانات محددة (مثل short). لماذا لا يمكن أن يحتوي الـ static class على instance constructors؟

لأن الـ static class لا يمكن أخذ نسخة (Instance) منه بالأساس، لذا فوجود مُنشئ للنسخ (Instance Constructor) ليس له أي معنى برمجي. ما هي مميزات استخدام Enum.TryParse مقارنة بالتحويل المباشر int.Parse؟

TryParse آمن ولا يرمي Exception في حالة الفشل (يتفادى الـ Runtime errors)، بل يعود بقيمة false، مما يجعل الكود أكثر استقراراً وسرعة. ما الفرق بين عمل override لـ Equals واستخدام == في الـ struct والـ class؟

في الـ Class: == الافتراضي يقارن الـ Reference (عنوان الذاكرة)، بينما Equals يمكن عمل override له لمقارنة القيم (Values).

في الـ Struct: == غير مدعوم افتراضياً ويجب كتابته يدوياً، بينما Equals يقارن القيم باستخدام الـ Reflection (مما يجعله بطيئاً ما لم يتم عمل override له). لماذا عمل override للـ ToString مفيد مع الـ custom classes؟

لأنه يوفر تمثيلاً نصياً مفهوماً للإنسان (Human-readable)، مما يسهل جداً عمليات الـ Debugging وعرض البيانات على واجهة المستخدم. هل يمكن تقييد الـ Generics لأنواع محددة في C#؟ أعط مثلاً.

نعم، باستخدام الـ Constraints where. مثال: `public void Print<T>(T item) where T : class`. ما هي الفروق الرئيسية بين الـ Generic Methods والـ Generic Classes؟

الـ Generic Class يعرف هيكل كامل (Class) يعتمد على نوع معين T لجميع خصائصه ودواله (مثل `List<T>`). بينما الـ Generic Method هي دالة واحدة داخل كلاس (عادي أو جينيريك) تقبل نوعاً عاماً لتنفيذ عملية معينة (مثل التبديل `Swap<T>`). لماذا يفضل استخدام `Generic swap method` عن عمل دوال مخصصة لكل نوع؟

لأنه يحقق مبدأ تقليل التكرار (DRY - Don't Repeat Yourself) ويعزز إعادة الاستخدام (Reusability). دالة واحدة تخدم كل الأنواع بشكل آمن برمجياً. كيف يؤدي عمل `override` للـ `Equals` في كلاس `Department` لتحسين دقة البحث؟

يضمن أن عملية البحث تقارن محتوى الـ `Department` (مثل الـ ID والاسم) بدلاً من مقارنة العناوين في الذاكرة (References)، مما يعيد نتائج صحيحة حتى لو كانا كائنين مختلفين في الذاكرة يحملان نفس البيانات. لماذا الـ `==` غير مُنفذ افتراضياً للـ `structs`؟

لتجنب الأداء البطيء الناتج عن استخدام الـ Reflection للبحث في كل خصائص الـ `Struct` للمقارنة. لغة `C#` تترك للمبرمج مسؤولية تعريف الـ `==` بالطريقة الأنسب والأسرع لحالته. ماذا نعني بمفهوم التعميم (Generalization) باستخدام `Generics`؟

هو كتابة الكود بشكل مجرد ومرن ليتمكن من التعامل مع أنواع بيانات مختلفة دون التضحية بأمان الأنواع (Type Safety)، مما يبني أنظمة قابلة للتوسع وأقل تكراراً. ماذا نعني بتصميم التسلسل الهرمي (Hierarchy design) في بيئة العمل الحقيقية؟

هو تنظيم الكيانات البرمجية لتعكس العلاقات في الواقع (مثلاً: `Employee` هو الأساس، ويتفرع منه `Manager` و `Developer`). هذا يساعد في ترتيب الـ `Business Logic`، تطبيق مبدأ الـ `Inheritance` بشكل صحيح، وسهولة التعديل مستقبلاً. ثالثاً: مقالة لينكد إن (LinkedIn Article): [Title](#)

Demystifying Class Types in C# 🚧 Ever wondered about the different types of classes in C# and when to use them? As a .NET developer, choosing the right class type is crucial for your application's architecture and performance. Here's a quick breakdown:

- 1 Regular Classes: The standard blueprint for creating objects. You can instantiate them, inherit from them, and build the core of your OOP structure.
- 2 Static Classes: Think of these as toolboxes. They cannot be instantiated or inherited. Perfect for utility functions (like Math operations or common helpers) where state doesn't need to be maintained across instances.
- 3 Sealed Classes: The end of the inheritance line! Use sealed when you want to strictly prevent any other class from inheriting from yours. It's a great practice for security and can even offer slight performance optimizations during runtime.
- 4 Abstract Classes: The foundational templates. You can't instantiate an abstract class directly. Instead, you use it to define common behaviors and force derived classes to implement specific methods using the abstract keyword. Ideal for implementing the "Is-A" relationship robustly.
- 5

Partial Classes: Splitting a single class across multiple files. Very common in UI frameworks or generated code (like Entity Framework) so developers can add custom logic without overwriting auto-generated code. Understanding these structures helps in writing cleaner, more maintainable code. Which class type do you find yourself using the most outside of the regular class? #CSharp #DotNet

#SoftwareDevelopment #ProgrammingTips #TechCommunity #OOP سؤال

سيلف ستادي وحله (Self-Study: Event-Driven Programming) سؤال سيلف ستادي: ما هي البرمجة الموجهة بالأحداث (Event-Driven Programming)؟ وكيف يتم تطبيقها؟ الحل: البرمجة الموجهة بالأحداث (Event-Driven Programming) هي نمط برمجي (Paradigm) يعتمد فيه تدفق البرنامج (Flow of control) بشكل أساسي على "الأحداث" (Events). كيف تعمل؟ بدلاً من أن يتم تنفيذ الكود بشكل تسلسلي بحت أو انتظار مدخلات المستخدم في سطر أوامر، البرنامج يظل في حالة "استماع" (Listening). عندما يحدث شيء معين (مثل: نقرة زر بالماوس، رسالة من السيرفر، استشعار لتغيير في قاعدة البيانات، أو انتهاء Timer)، يتم إطلاق "حدث" (Event trigger). هذا الحدث يقوم بتشغيل دالة معينة مخصصة للرد عليه تُسمى (Event Handler). أهميتها واستخداماتها: تطبيقات الواجهات الرسومية (GUI): مثل تطبيقات الويندوز أو الويب. المستخدم يضغط على زر "Submit"، هذا الضغط هو الـ Event الذي يشغل كود رفع البيانات. الأنظمة الحديثة وتطبيقات الوقت الفعلي (Real-time systems): مثل برامج الشات، أو إنترنت الأشياء (IoT). فصل المكونات (Decoupling): المكون الذي يُصدر الحدث (Publisher) لا يحتاج أن يعرف أي تفاصيل عن المكون الذي يستقبل الحدث (Subscriber).